IFT 211: Digital Logic Design

Lecture 2: Boolean Algebra

2.1 Boolean algebra

What is Boolean algebra?

Boolean algebra is the algebra of logic. It deals with variables that can have two discrete values, 0 (False) and 1 (True); and operations that have logical significance. This earliest method of manipulating symbolic logic was invented by George Boole and subsequently came to be known as Boolean algebra. Boolean algebra has now become an indispensable tool in computer science for its wide applicability in switching theory, building basic electronic circuits, and designing digital computers.

2.1.1 Fundamental concepts of Boolean algebra

i. Use of Binary Digit

Boolean equations can have either of two possible values, 0 and 1

ii. Logical Addition

Symbol '+', also known as 'OR' operator, used for logical addition. Follows law of binary addition

iii. Logical Multiplication

Symbol '.', also known as 'AND' operator, used for logical multiplication. Follows law of binary multiplication

iv. Complementation

Symbol '-' or '' ', also known as 'NOT' operator, used for complementation. Follows law of binary compliment.

2.1.2 Operator Precedence

Each operator has a precedence level, the higher the operator's precedence level, the earlier it is evaluated in an expression. Expressions are scanned from left to right. Expressions enclosed within parentheses are evaluated first, then, all complement (NOT) operations are performed, followed by all '.' (AND) operations and all '+' (OR) operators.

Some major difference between Boolean algebra and ordinary algebra

1. Boolean algebra applies to finite set of elements {0 and 1}, while ordinary algebra would apply to infinite set of real values.
2. Complement is available in Boolean algebra , but not in ordinary algebra
3. There are no subtraction or division operations in Boolean algebra.
4. The values assigned to Boolean variables have logical significance, while the values assigned to ordinary algebra variables have numerical significance.

2.1.3 Principle of Duality

The principle of duality is a very import concept in Boolean algebra. Briefly stated, the principle of duality states that;

If an expression is valid in Boolean algebra, the dual of the expression is also valid. The dual expression is found by replacing all OR (+) with AND (.), all AND with OR, all ones with zeros, and all zeros with ones.

The principle of duality will be extensively used in proving Boolean algebra theorems. In fact, once we have employed the postulates of previously proven theorems to demonstrate the validity of one expression, duality can be used to prove the validity of the dual expression.

2.1.4 Postulates of Boolean algebra

Boolean algebra can be used to solve complex expressions in order to simplify a logic circuit. Most modern designs and nearly all complex logic circuits are internal to ICs. Boolean algebra has several basic rules called postulates or identities, which are used extensively in proving Boolean algebra theorems and simplifying complex logic circuits.

Postulate 1:

(a) A = 0, if and only if, A is not equal to 1

(b) A = 1, if and only if, A is not equal to 0

Postulate 2:

(a) A+ 0 = A

(b) A. 1 = A

Postulate 3: Commutative Law

(a) A + B = B + A

(b) A. B = B. A

Postulate 4: Associative Law

(a) A + (B + C) = (A + B) + C

(b) A. (B. C) = (A. B). C

Postulate 5: Distributive Law

(a) A + (B. C) = (A + C). (A+ C)

(b) A. (B + C) = (A. B) + (A. C)

Postulate 6:

(a) $A + A' = 1$

(b) $A.A' = 0$

2.1.5 Theorems of Boolean algebra

We shall now state several useful theorems in Boolean algebra. In these theorems, the latter a,b,c,… represent elements of a Boolean algebra. The first theorem describes the property of idempotency and is stated as follows.

Theorem 1.

(a) $a + a = a$

(b) $a . a = a$

Proof:

We may prove either part (a) or (b) of the theorem. Suppose we want to prove part (a)

$a + a = a$

$a + a$	$= (a + a).1$	[P2b]
	$= (a + a)(a + a')$	[P6a]
	$= a + a.a'$	[P5a]
	$= a + 0$	[P6a]
	$= a$	[P2a]

The postulates used to justify a particular step are listed to the right.

Theorem 2.

(a) $a + 1 = 1$

(b) $a . 0 = 0$

Proof. Let us again prove part (a) of the theorem.

$a + 1 = (a + 1) . 1$ [P2b]

$= 1 .(a + 1)$ [P3b]

$= (a + a')(a +1)$ [P6a]

$= a + a' .1$ [P5a]

$= a + a'$ [P2b]

$= 1$ [P6a]

Since part (a) of the above theorem is valid, it follows from the principle of duality that part (b) is also valid.

Theorem 3.

$a'' = a$

proof.

a' is the compliment of **a** and also a is the complement of a'. Since the compliment of a' is unique, it follows that $a'' = a$

Theorem 4

(a) $a+ab = a$

(b) $a(a+b) = a$

Proof.

Let us prove part (a)

$a + ab = a.1 + ab$ [P2b]

$= a(1 + b)$ [P5b]

$= a(b+1)$ [P3b]

$= a . 1$ [T2a]

$= a$ [P2b]

Theorem 5

(a) $a + a'b = a\ b$

(b) $a(a'+b) = ab$

Proof.

Part (a) of the theorem is proved as follows:

$a + a'b = (a + a')(a + b)$ [P5a]

$= 1 . (a + b)$ [P6a]

$= (a + b). 1$ [P3b]

$= a+b$ [P2b]

Theorem 6

(a) $ab + ab' = a$

(b) $(a+b)(a+b') = a$

Proof. Part (a) of the theorem is proved as follows:

$ab + ab' = a(b+b')$ [P5b]

$= a . 1$ [P6a]

$= a$ [P2b]

Theorem 7

(a) $ab + ab'c = ab + ac$

(b) $(a+b)(a+b'+c) = (a+b)(a+c)$

Proof.

Part (b) of the theorem is proved as follows:

$(a + b)(a + b' + c) = a + \{b(b' + c)\}$ [P5a]

$= a + (b.b' + b.c)$ [P5b]

$= a + (0 + b.c)$ [P6b]

$= a + (b.c + 0)$ [P3a]

$= a + (b.c)$ [P2a]

$= (a + b) (a + c)$ [P5a]

Theorem 8. DeMorgan's Theorem.

(a) $\overline{a + b} = \overline{a}.\overline{b}$

(b) $\overline{a.b} = \overline{a} + \overline{b}$

2.2 Boolean Functions

A Boolean function is an expression formed with binary variables, operators (OR, AND & NOT), parentheses, and the equality sign. The value of a Boolean function can be either 0 or 1. A Boolean function may be represented as an algebraic expression or a truth table.

1. Representation as an algebraic expression

$F(x,y) = x .y + x.y'$

The right-hand side of the equation is called an expression, and the symbols x,y,z are the literals of the function. For a given Boolean function, there may be more than one algebraic expression.

2. Representation as a truth table

Another way to represent a Boolean function is by the use of a truth table. If we evaluate a Boolean function for all possible input combinations and list the results in tabular form, we obtain a unique representation of the function called a truth table. For example, if we evaluate the function F(x,y) for all possible input combinations and list them in tabular form, we obtain the truth table of the function F(x,y).

X	Y	F
0	0	0
0	1	1
1	0	0
1	1	1

The number of rows in a truth table is equal to 2^n where n is the number of literals in the function. The combination of 0s and 1s for rows of this table are obtained from binary numbers by counting from 0 to $2^n - 1$.

Examples of Boolean functions

i. F(a,b) = a + b

ii. F(a,b) = a.b

iii. F(a,b) = a' + b

Exercise

1. Prepare a truth table for the above Boolean functions.

2. Prepare a truth table for the following Boolean functions

(i) $F(A,B,C) = AB + A'B + AC'$

(ii) $F(A,B,C) = AB + A'C + AC$

2.2.1 FORMS OF BOOLEAN Function

In digital logic design, Boolean functions represent logical relationships between binary variables. Understanding the different forms of Boolean functions is crucial for designing and simplifying digital circuits. The primary forms are the **Sum of Products (SOP)** and **Product of Sums (POS)**, along with their canonical forms.

Sum of Product (SOP)

Product/AND Term: A product term is a term consisting of a product of literals (variable).

e.g.

I. A.B
II. A.B.C
III. A.B.C.D

When two or more product terms are logically added (ORED) together, the resulting expression is called the sum of products. First, the product terms are formed, then these are summed (OR). i.e. SOP is a products term that are logically added together.

e.g.

I. $F(A,B,C) = AB + A'BC + A'B'C'$
II. $F(A,B,C) = A'.B'.C' + A.B'.C + AB$

Minterm

Minterm is a product term that include all the input variables (literals) of a function either in their normal or complemented form.

e.g.

$F(A, B) = AB + A'B + A$

In the above function, the first two terms AB and A'B are minterms.

Product of Sum (POS)

Sum term: Sum term is a term consisting of a sum of literals either complimented or complimented

e.g.

I. A
II. A+B
III. A+B+C

Product of Sum

When two or more sum terms are logically multiplied (ANDED) together, the resulting expression is called the product of sum. First, the sum terms are formed, and then the products are taken.

e.g.

i. F(A,B,C) = A.

(A+B).(A+B+C').(A+B'+C') ii. F(A,B,C) =

(A+B').(A+B'+C').(A+C)

A POS expression can contain a single variable term, as in (i)

Maxterm

Maxterm is sum term that include all the input variables of a function either in their normal form or complemented form.

e.g.

F (A, B) = A. (A+B). B

In the above function, the term (A+B) is a **maxterm**

Canonical/standard form of Boolean function

Canonical form is not efficient, but sometimes it is useful for design and analysis. A Boolean function is said to be in canonical form if every variable in the domain of the expression appears in every term in the expression.

For example, the following Boolean functions are in their standard forms

F(A,B) = A’B’ + AB - canonical SOP

F(A,B,C) = (A+B+C). (A’+B’+C).(A’+B+C’) - Canonical POS

Canonical SOP

A Boolean function is said to be in canonical or standard SOP form if it is constructed (formed) using only minterms.

For example,

F (A, B) = A’B’ + AB’ + A’B,

is in canonical SOP form as all it product terms are minterms.

An SOP expression can be forced into cononical form by ANDing the incomplete terms with terms of the form (X+X’) where X is the name of the missing variable

For example,

F(A,B,C) = AB + BC

= AB(C+C’) + BC(A+A’)

= ABC + ABC’ + ABC + A’BC

= ABC + ABC’ + A’BC

Canonical POS

A Boolean function is said to be is canonical or standard POS form if it is constructed or formed using only maxterms.

For example,

F (A, B, C) = (A+B+C) (A’+B+C) (A+B+C’)

is in canonical POS form as all its sum terms are maxterms.

IFT 211: Digital Logic Design

Lecture 3: Minimization of Boolean Functions

3. MINIMIZATION OF BOOLEAN FUNCTIONS

Simplification Goal:

A simplified Boolean function can be realized with fewer electronic components, resulting in a cheaper circuit.

The simplification of a switching function is a common and worthwhile goal. Its importance stems from the fact that the simpler the function, the easier it is to realize. The goal of the simplification is to minimize the cost of realizing a function with physical circuit elements, where the definition of cost depends on the nature of the circuit elements to be used. In general, it is desirable to minimize the number of circuit elements and to make it as simple as possible. In a two-level sum of product realization of a switching function, minimizing cost implies reducing the number of product terms in the expression representing the function (to reduce gate count) and minimizing the number of literals in each product term (to minimize gate complexity) measured here in terms of the number of gate inputs.

3.1 Methods of Minimizing or Simplifying Boolean Functions

1. Using Boolean algebra (postulates or theorems)
2. Using Karnaugh Map
3. Quine- McCluskey Method

3.1.1 Minimization or Simplification of Boolean Expression using Boolean Algebra postulates and theorems

There are several methods to minimize the Boolean function. Here, simplification or minimization of complex algebraic expressions will be shown with the help of postulates and theorems of Boolean algebra.

Example 1. ***Minimize the Boolean function F=AB+ BC + B'C.***

Solution.

F = AB + BC + B'C

= AB + C(B + B') ---- P5b

= AB + C ----P6a

Example 2. ***Minimize the Boolean function F= A + A'B.***

Solution. F = A+ A'B

= (A + A') (A + B) ----p5a

= A + B ---p6a

Example 3. ***Minimize the Boolean function F= A'B'C + A'BC + AB'.***

Solution. F = A'B'C + A'BC + AB'

= A'C (B'+B) + AB' ----P5b

= A'C + AB' ----P6a

Example 4. ***Minimize the Boolean function F = AB + (AC)' + AB'C(AB + C).***

Solution. F = AB + (AC)' + AB'C(AB + C)

= AB + A' + C'+ AB'C.AB + AB'C.C

= AB + A' + C' + 0 + AB'C *(B.B' = 0 and C.C = C)*

= ABC + ABC' + A' + C' + AB'C *(AB = AB(C + C') = ABC + ABC')*

= AC(B + B') + C'(AB + 1) + A'

= AC + C'+A' *(B + B' = 1 and AB + 1 = 1)*

= AC + (AC)' = 1

Example 5. Minimize the Boolean function $f\ (x, y, z) = \bar{x}y(z + \bar{y}x) + \bar{y}z$

Solution

$$f = \bar{x}yz + \bar{x}y\bar{y}x + \bar{y}z \text{ ---------- } p5b$$

$$= \bar{x}yz + o + \bar{y}z \text{ ---------- } p6b$$

$$= \bar{x}yz + \bar{y}z \text{ ---------- } p2a$$

$= \bar{x}z + \bar{y}z$ ---------- $T7a$

$= (\bar{x} + \bar{y}) + z$ ---------- $p5b$

The last two forms represent the minimum sop and POS forms, respectively. The minimum SOP form has two terms and a total of four literals. It therefore requires two input AND gates and one two-input OR gate to realize.

Exercise

Minimize the following Boolean Function

1. $F(A,B) = A'B' + A'B + AB' + AB$
2. $F(A,B,C) = A'B'C + A'BC + AB'C + ABC$
3. $f(A,B,C,D) = ABC + ABD + \bar{A}\,B\,\bar{C} + CD + B\bar{D}$
4. $f(A,B,C) = \acute{A}\,BC + A\acute{B}C + ABC$
5. $f(A,B,C) = ABC + A\bar{B}C$
6. $f(A,B,C,D) = (AD + B + C)(AD + (\overline{B + C}))$

3.1.2 Minimization of Boolean Expression using K. Map

A Karnaugh map provides a systematic method for simplifying Boolean expressions and, if properly used, will produce the simplest SOP or POS expression possible, known as the minimum expression. As you have seen, the effectiveness of algebraic simplification depends on your familiarity with all the laws, rules, and theorems of Boolean algebra and on your ability to apply them. The Karnaugh map, on the other hand, provides a "cookbook" method for simplification. A Karnaugh map is similar to a truth table because it presents all of the possible values of input variables and the resulting output for each value. Instead of being organized into columns and rows like a truth table, the Karnaugh map is an array of cells in which each cell represents a binary value of the input variables. The cells are arranged in a way so that simplification of a given expression is simply a matter of properly grouping the cells. Karnaugh maps can be used for expressions with two, three, four. and five variables. Another method, called the Quine-McClusky method can be used for higher numbers of variables. The number of cells in a Karnaugh map is equal to the total number of possible input variable combinations as is the number of rows in a truth table. For three variables, the number of cells is $2^3 = 8$. For four variables, the number of cells is $2^4 = 16$.

The 3-Variable Karnaugh Map

The 3-variable Karnaugh map is an array of eight cells. as shown in the following figure. In this case, A, B, and C are used for the variables, although other letters could be used. Binary values of A are along the left side (notice the sequence), and the values of B and C are across the top. The value of a given cell is the binary values of A at the left in the same row, combined with the value of B and C at the top in the same column. For example, the cell in the upper left corner has a binary value of 000, and the cell in the lower right corner has a binary value of 110. The following figure shows the standard product terms that are represented by each cell in the 3-variable Karnaugh map

A \ BC	00	01	11	10
0	A'B'C'	A'B'C	A'BC	A'BC'
1	AB'C'	AB'C	ABC	ABC'

Cell Adjacency

The cells in a Karnaugh map are arranged so that there is only a single variable change between adjacent cells. Adjacency is defined by a single variable change. Cells on edges are considered adjacent to cells on opposite edges. In the 3-variable map the 010 cell is adjacent to the 000 cell, the 011 cell, and the 110 cell. The 010 cell is not adjacent to the 001 cell, the 111 cell, the 100 cell, or the 101 cell.

A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

Steps for Minimizing Boolean Functions Using K-Maps

- **Determine the Number of Variables**
  - Identify the number of variables in the Boolean function.
  - This dictates the size of the K-map:
    - 2 variables: 2x2 grid
    - 3 variables: 2x4 grid
    - 4 variables: 4x4 grid
    - And so on.
- **Draw the K-Map**
  - Create a grid based on the number of variables.
  - Label the rows and columns using Gray code (binary sequence where two successive values differ by only one bit).
- **Populate the K-Map**
  - Fill in the K-map with the values from the Boolean function's truth table.
  - Place a 1 in the cell corresponding to each minterm where the function is true.
  - Optionally, place 0s for false minterms and Xs for "don't care" conditions.
- **Group the 1s**
  - Form groups (clusters) of 1s in the K-map.
  - Each group should be a rectangle and contain 1, 2, 4, 8, etc. cells (powers of 2).
  - Groups can wrap around the edges of the K-map.
  - Overlapping groups and including each 1 in multiple groups is allowed if it results in larger groups.
- **Determine the Product Terms for Each Group**
  - For each group, determine the product (AND) term:
    - A variable is included in the product term if it is the same for all 1s in the group.
    - A variable is omitted if it changes within the group.
    - A variable appears complemented if it is 0 in all members of the group and uncomplemented if it is 1.
- **Combine the Product Terms**
  - Write the simplified Boolean function as the OR (sum) of all the product terms obtained from the groups.

Two-Variable Karnaugh Maps Example

A two-variable Karnaugh map is shown in Figure (3).

X	Y	Minterms	
0	0	x'y'	m0
0	1	x'y	m1
1	0	xy'	m2
1	1	xy	m3

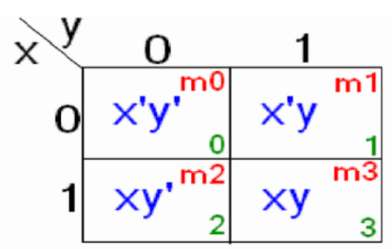
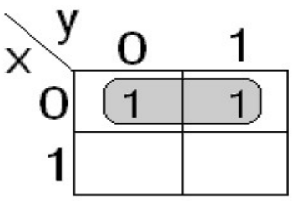


Figure (3) : Two variable karnaugh map.

Example 1: Simplify the following Boolean expression using Karnaugh map: F(x,y) =

x'y+x'y'

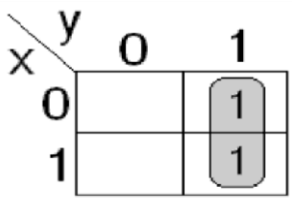
Solution:



F(x,y) = x'

Example 2: Simplify the following Boolean expression using Karnaugh map: F(x,y) = **xy+x'y**

Solution:

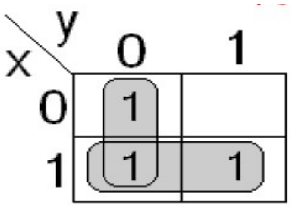


F(x,y) = y

Example 3: Simplify the following Boolean expression using Karnaugh map: F(x,y) =

x'y'+xy'+xy

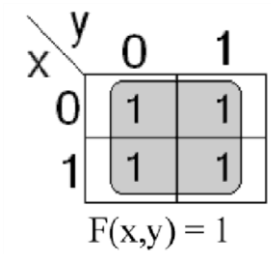
Solution:



F(x,y) = x + y'

Example 4: Simplify the following Boolean expression using Karnaugh map: F(x,y) =
xy+x'y+xy'+x'y'

Solution:



Three-Variable Karnaugh Maps Examples

A three-variable Karnaugh map is shown in Figure (4).

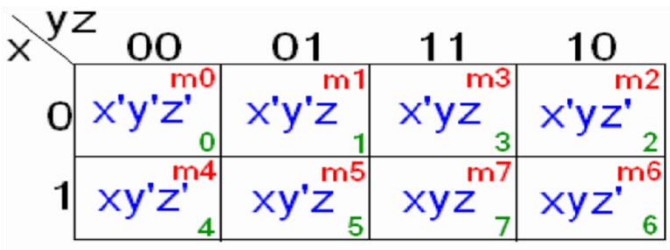
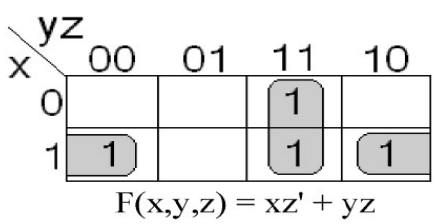


Figure (4) : Three variable karnaugh map.

Example 5: Simplify the following Boolean expression using Karnaugh map: F(x,y,z)=

Σ(3,4,6,7)

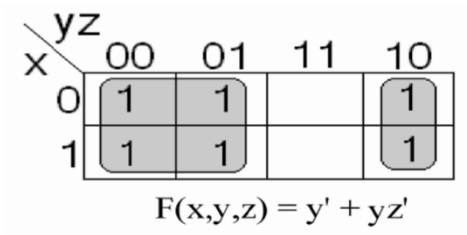
Solution:



Example 6: Simplify the following Boolean expression using Karnaugh map: F(x,y,z)=

Σ(0,1,2,4,5,6)

Solution:



Four-Variable Karnaugh Maps Examples

A four-variable Karnaugh map is shown in Figure (5).

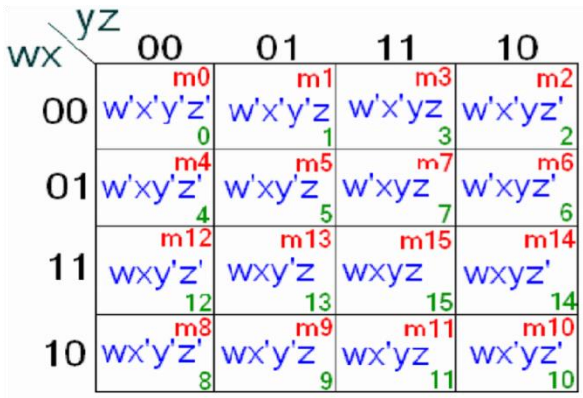
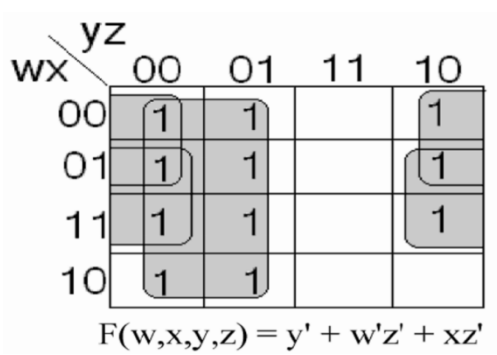


Figure (5) : Four variable karnaugh map.

Example 7: Simplify the following Boolean expression using

Karnaugh map: F(w,x,y,z)= **Σ(0,1,2,4,5,6,8,9,12,13,14)**

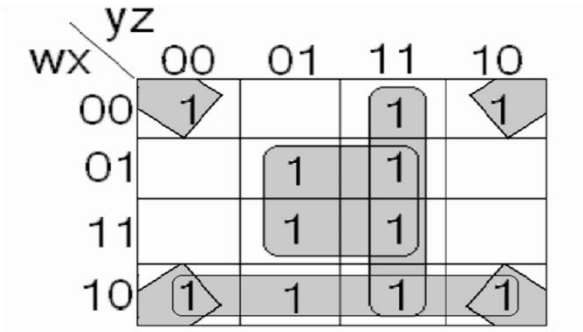
Solution:



Example 8: Simplify the following Boolean expression using Karnaugh

map: F(w,x,y,z)= **Σ(0,2,3,5,7,8,9,10,11,13,15)**

Solution:



F(w,x,y,z) = wx' + yz + xz + x'z'

Exercise

Minimize the following Boolean Expression using K . Map

1. $f(A,B,C) = \sum m\,(1,3,5,7)$
2. $f(A,B,C) = \sum m\,(0,1,2,4,7)$
3. $f(A,B,C) = \sum m\,(0,1,\ 5,6,7)$
4. $f(A,B,C,D) = \sum m(0,1,2,3,7,11,13,14,15)$
5.

x y z	
000	0
001	0
010	1
011	1
100	1
101	1
110	1
111	1

a. Write the corresponding Boolean function
b. Minimize the Boolean function
c. Draw the circuit diagram of the minimized function